[A] Managing Marriages and Divorces

```
Time limit:   3 seconds
Memory limit: 65535 kBytes
```

Description

The secret service of Landia decided to process all marriage and divorce requests to eliminate any possible errors.

Input

The input is structured as follows:

- The first line contains the number of persons, $1 \leq np \leq 250,000$.
- Each of the following np lines contains information about one person: a personal code (a 31-bit positive integer) and their sex (M or F).
- The remaining lines (an unspecified number) contain requests in one of the following formats: `M code1 code2` – a marriage request; `D code1 code2` – a divorce request.

It is guaranteed that code1 and `code2` are valid, meaning they correspond to existing persons in the input list.

General rules (applied to every request in this strict order):

- if code1 does not belong to a man, error code 1 is returned
- if code2 does not belong to a woman, error code 2 is returned

If a marriage request is accepted, the two persons become married. Record the partner code for each spouse

Marriage requests:

- if code1 belongs to a married man, error code 3 is returned
- if code2 belongs to a married woman, error code 4 is returned

Divorce requests:

- if the two codes do not correspond to a married couple, error code 5 is returned
- if a divorce request is accepted, the two persons become unmarried. Erase the partner information for both.

At the start, every person is unmarried. The number of marriage and divorce requests does not exceed 200,000.

Output

Display the number of valid marriage (`M`) requests, the number of valid divorce (`D`) requests, and the number of error codes 1 through 5. All five error counts must be displayed, even if the corresponding count is zero.

Example

Input

```
6
101 M
201 F
121 M
221 F
301 M
311 F
M 101 201
M 121 301
M 121 221
M 201 101
M 301 221
M 121 201
D 101 201
D 101 221
D 101 201
M 101 311
```

Output

```
Valid M: 3
Valid D: 1
Error code 1:  1
Error code 2:  1
Error code 3:  1
Error code 4:  1
Error code 5:  2
```

[B] Booster Rush

Time limit: 3 second
Memory limit: 65535 kBytes

Description

Nicole's favorite computer game is a racing game where the player must complete a track of length L by driving their car as fast as possible.

Along the track, there are N identical *boosters*, numbered from 0 to $N-1$. Without any booster, the car moves at its normal speed of A units per second. If the player collects a booster, the following occurs:

- It takes T seconds to apply the newly collected booster. During this period, the car continues moving at the normal speed A. If the car was already under the effect of another booster when picking up a new one, the previous booster is discarded, and the application process for the new booster begins.
- After T seconds, the booster activates, increasing the car's speed to B units per second for the next R seconds.
- After these R seconds, the booster effect ends, and the car's speed returns to A.

Note that a booster can be replaced by a newly picked-up booster at any of these three stages.

Nicole knows the track very well and is familiar with the position of every booster. Booster i is located P_i units away from the start.

Help Nicole determine which boosters she should pick up to finish the track in the shortest possible time.

Input

The first line of input contains five integers:

- L ($1 \leq L \leq 10^{11}$) – the total length of the track;
- A, B ($1 \leq A < B \leq 200$) – the car's normal speed and boosted speed, respectively;
- T ($0 \leq T \leq 300$) – the time required to apply a new booster;
- R ($1 \leq R \leq 1200$) – the duration for which the booster remains active.

The second line contains an integer N ($0 \leq N \leq 500\,000$) – the number of boosters placed along the track.

The third line contains N integers P_i ($0 \leq P_i \leq L$) – the distances of the boosters from the start of the track. These values are given in strictly ascending order, and all are unique.

Output

Output two lines:

- The first line should contain the number of boosters Nicole needs to pick up to complete the track in the shortest possible time.
- The second line should contain the indices of the boosters she should pick up.

The indices may be printed in any order, but each index must appear at most once.

If multiple optimal solutions exist, any of them may be printed. Your solution will be accepted if the total completion time using the chosen boosters differs from the optimal time by at most 10^{-9} in absolute or relative error.

Example

Input	Output
100000 100 138 60 300 5 5000 20000 50000 55000 75000	2 0 3
100000 78 86 9 560 4 13505 69705 87448 92090	2 0 1

Explanation

The events in the sample output of the first example are as follows:

- The car travels the first 5000 units at normal speed and picks up the first booster. It continues moving at normal speed until it reaches the 11000-unit mark from the start.
- The booster then activates, allowing the car to travel 41400 units at boosted speed. The booster depletes at a total distance of 52400 units from the start. Note that the car passes by two additional boosters without collecting them.
- The car continues for another 2600 units at normal speed before reaching (and picking up) booster 3 at the 55000-unit mark. While the new booster is being applied, the car travels an additional 6000 units at normal speed.
- The second booster activates at 61000 units from the start. Although it could provide boosted speed for 41400 units, the car reaches the finish line after traveling only 39000 of those units.

The total time to complete the track is $50+60+300+26+60+\frac{39000}{138}$ seconds, and it can be shown that this result is optimal.

Note that the configuration shown in the sample output is not unique: picking up booster 2 instead of booster 3 yields exactly the same total completion time.

[C] Favorite Scene

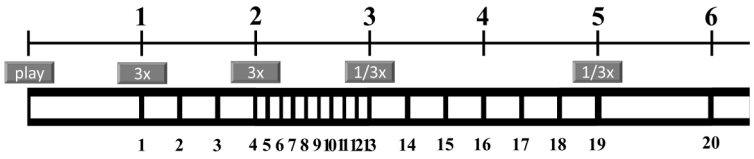
Time limit: 2 second
Memory limit: 65535 kBytes

Description

Your mom often watches old video tapes of her favorite classic movies. She has watched them so many times that she knows the exact starting times of her favorite scenes. Today, she wants to show you one of these scenes. Knowing that the attention span of modern youth is quite short, she wants to fast-forward the tape to the correct moment as quickly as possible.

When the [play] button is pressed, the film starts at normal playback speed (playing 1 second of footage in 1 second of real time). The player has two additional buttons to control playback speed: the [3x] button triples the current speed, while the [1/3x] button reduces it to one third. You can press at most one button per second. Exactly one second after playback begins – and every second thereafter – the player checks for any button presses and updates the playback speed accordingly (either leaving it unchanged, tripling it, or reducing it to one third).

For example, assume that the favorite scene starts at 19 seconds from the start of the film. By pressing the [3x] button in the first and second seconds, and the [1/3x] button in the third and fifth second, the desired scene can be watched at normal speed five seconds after the start.



Your task is to determine the shortest possible time from the start of playback to the beginning of the desired scene. Playback of the desired scene must begin at normal speed.

Input

The input is a single line with an integer T – the start time of the favourite scene.

Output

Print the minimum possible time (in seconds) before you can start watching the target scene at normal speed.

Constraints

- $0 \leq T < 2^{50}$

Example

Input	Output
19	5
123456789098765	85
3	3
4	2

[D] The Great Escape

Time limit: 2 second
Memory limit: 65535 kBytes

Description You're a tiny, adventurous guppy named Gill, living in a vibrant, three-dimensional aquarium. This isn't your average glass box; it's a massive, cuboid structure with special "water zones" that have different energy costs to swim through.

Your journey starts at the origin of the aquarium, at coordinates **(0, 0, 0)**. Your goal is to reach your favorite hiding spot, a cozy little coral at coordinates **(N-1, M-1, P-1)**.

The aquarium's dimensions are **N x M x P**. The energy cost for each zone varies depending on its coordinates **(x, y, z)**, the dimensions of the aquarium, and an aquarium constant **K**. It's calculated using a mysterious formula:

$$\textbf{Cost}(x, y, z) = (x \cdot y \cdot z + N + M + P) \pmod{K},$$
$$where 0 \leq x < N, 0 \leq y < M, 0 \leq z < P$$

Being a guppy, you can only swim in three specific directions from your current location **(x, y, z)**:

- **Move forward:** to **(x+1, y, z)**
- **Move to the side:** to **(x, y+1, z)**
- **Move upward:** to **(x, y, z+1)**

Task

Your task is to find a path from **(0, 0, 0)** to **(N-1, M-1, P-1)** that minimizes the total energy cost (sum for all visited cells, including both the start and the destination). This means you need to navigate through the aquarium's grid, accumulating the minimum possible cost.

Input

The input consists of a single line containing four integers: `N M P K`.

Output

Print a single integer representing the total minimum energy cost required for the journey.

Constraints

- $1 \leq N, M, P \leq 100$
- $1 \leq K \leq 1,000,000$

Example

Input	Output
2 2 2 100	25
2 2 2 4	9
13 17 19 23	175

[E] String Inversions

Time limit: 1 second
Memory limit: 65535 kBytes

Description

Ivan came up with an interesting idea for a programming contest problem: given a string S consisting of lowercase English letters, count the number of *inversions* in it. An inversion is a pair of indices $i < j$ such that the letter S_i appears later in the alphabet than S_j.

However, other jury members reminded Ivan that there are already many well-known algorithms for counting inversions. To make the problem more original, Ivan suggested the following modification: in addition to S, a positive integer N is given, and the task is to compute the number of inversions in $N \cdot S$ – the string formed by repeating S exactly N times.

The jury is now curious how to solve this new version of the problem. Help them by finding the solution.

Input

The first line of the input contains the string S consisting of lowercase English letters.

The second line contains the integer N – the number of repetitions of S.

Output

Output a single line with the number of inversions in the repeated string. This number can be large, so print it modulo $10^9 + 7$.

Constraints

- $1 \leq |S| \leq 100\,000$
- $1 \leq N \leq 10^{12}$

Example

Input	Output
ba 1	1
ab 3	3
zkba 1	6
cab 7	77

[F] Circle Packing

Time limit: 1 second
Memory limit: 65535 kBytes

Description

There are six unit-diameter disks. Arrange the disks in the plane so that no two overlap, and the size of the smallest square covering all of them is minimized.

Compute and report this minimum size.

Input

There is no input for this problem.

Output

Output a real number representing the side length of the square.

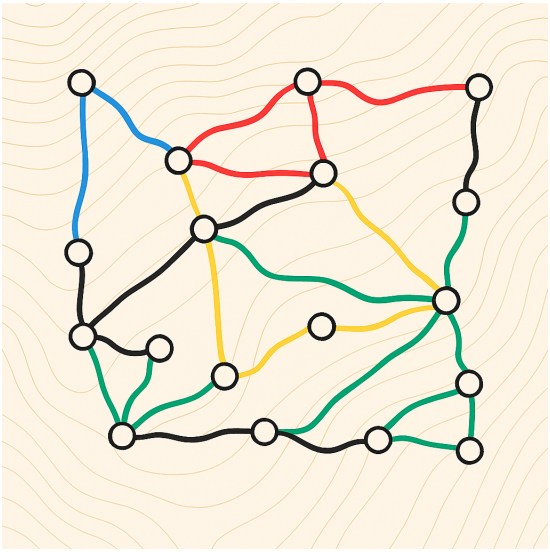
Print your answer rounded to six decimal digits, in the format `X.XXXXXX`.

[G] Training Plans

Time limit: 1 second
Memory limit: 65535 kBytes

Description

On orienteering maps, the junctions are connected by sections marked with blue, red, yellow, and green colors. The colors indicate the difficulty of each section.



On the map, countless acyclic routes can be designated, each of which can be described with the sequence of colors of the successive sections that form them. In the description of a route, each section has its own color. No section of the route is left out of the description, even if the route contains several sections of the same difficulty in direct succession. You can use one given color sequence to describe multiple such routes.

Your daily training plan is described by another sequence of colors. The description of a training plan never includes two identical colors in a row. At the same time, one letter in your training plan represents the longest possible contiguous sequence of sections of the given difficulty on the route. For example, if you are at an X in your training plan, you will complete this part of the plan when you start from your current position on the route and go through all the consecutive sections of difficulty X.

Your Task

When you choose a route for your daily training, you endeavor to complete your daily training plan as many times as possible on the chosen route. Your

task is to determine the maximum number of times you can do this for a given route.

Input

The input begins with a positive integer N on a line by itself, representing the number of test cases ($1 \leq N \leq 100$).

The first line of each test case is a string of up to 10000 characters, which contains exclusively the letters B (blue), R (red), Y (yellow), and G (green). This string describes a route on an orienteering map.

The next line consists of an integer M, representing the number of your training plans.

Each of the following M lines describes a training plan. The training plans also contain exclusively the letters B (blue), R (red), Y (yellow), and G (green). The total number of letters in all training plans for one test case does not exceed 32.

Output

For each test case, write the text "Test #*number*:" in the first line of the standard output, where *number* should be the sequence number of the test case. Start the numbering of the test cases from 1.

In the following lines, print the color sequence describing each training plan and the number of times that training plan can be completed on the route given in the test case. Use the exact format given in the sample output.

The output lines for two consecutive test cases should be separated from each other by a blank line.

Example

Input

```
5
BYGRBYGRBYGR
3
YG
GBR
GRBYGRBYG
GBBRRRYYYYGGBBBRRY
3
GBRY
RY
YG
YAYAYAYAY
2
AYA
Y
```

```
BBBBBBB
1
B
ABBAAABBBBAAAAA
1
ABA
```

Output

```
Test #1:
YG: 3
GBR: 0
GRBYGRBYG: 1

Test #2:
GBRY: 2
RY: 2
YG: 1

Test #3:
AYA: 2
Y: 5

Test #4:
B: 1

Test #5:
ABA: 1
```

[H] Pizza

```
Time limit:   2 second
Memory limit: 65536 kBytes
```

Description

John works at Pizza Food, a pizza delivery company. He has a car equipped with a trunk that can hold three columns of pizza boxes, numbered 1, 2, and 3.

Each pizza portion is packed in a square box, whose size is represented by a natural number. Boxes may have equal or different sizes.

When preparing deliveries, the staff load John's car by stacking the boxes into the three columns, one box on top of another, in no particular order.

John, however, prefers order. He wants to rearrange all the boxes into a single column, sorted in descending order (largest box at the base, smallest on top). To make the task more interesting, he follows these rules:

- Only the top box of any column may be moved.
- A box can only be placed on top of another column.
- A larger box cannot be placed on a smaller one.
- A box may always be placed on an empty column.

John's goal is to find a sequence of moves that transforms the initial configuration of the three columns into a single target column C, arranged in descending order by box size. In one move, he takes a box from a column $c1$ and moves it to some another column $c2$.

Given the initial arrangement of the boxes across the three columns and the target column C, find a sequence of moves that solves the problem.

Input

The input has the following structure:

- The first line contains the natural number n_1, representing the number of boxes on the first column. On the same line follow n_1 natural numbers, separated by a space, representing the sizes of the boxes on column 1 from the base to the top. If $n_1 = 0$, then there are no boxes on this column.
- The second line contains the natural number n_2, representing the number of boxes on the second column. On the same line follow n_2 natural numbers, separated by a space, representing the sizes of the boxes on column 2 from the base to the top. If $n_2 = 0$, then there are no boxes on this column.
- The third line contains the natural number n_3, representing the number of boxes on the third column. On the same line follow n_3 natural numbers, separated by a space, representing the sizes of the boxes on column 3 from the base to the top. If $n_3 = 0$, then there are no boxes on this column.

- The fourth line contains the natural number C, with possible values 1, 2, or 3, representing the column on which all the boxes must finally be gathered.

Output

The output has the following structure:

- On each line of the file, one move will be written, in the order in which they are performed. Each move consists of a pair of natural numbers c_1 c_2, meaning: the box from the top of column c_1 is moved onto column c_2.
- On the last line of the output file, the pair of numbers 0 0 will be printed.

Constraints

- $1 \leq c_1, c_2, C \leq 3$
- $20 \leq box\ size \leq 50$
- $2 \leq n_1 + n_2 + n_3 \leq 18$. That is, there are at least two boxes and at most 18.
- The number of moves in your solution must not exceed 3 000 000.
- There is always at least one sequence of moves that solves the problem. If multiple solutions exist, you may output any valid one. The number of moves does not need to be minimal.

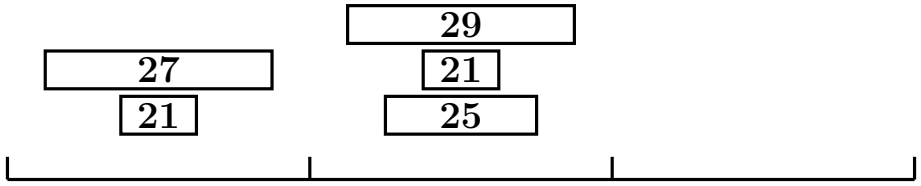
Example

Input	Output	Notes
2 21 27 3 25 21 29 0 3	2 3 1 3 2 1 2 3 1 2 1 3 2 3 0 0	This solution is not unique. The interpretation of the moves is explained below.

Explanation:

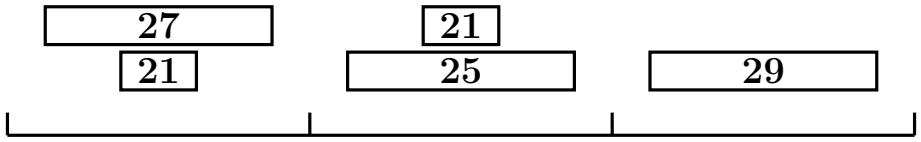
(The solution does not use the minimum number of steps)

Initial configuration

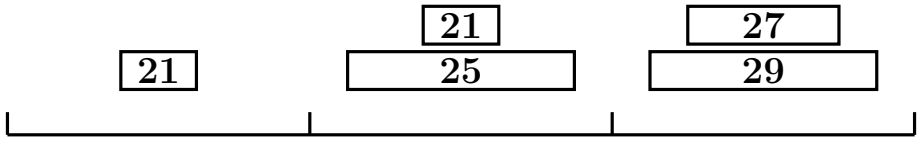


We have $C = 3$, which means that the boxes will be gathered on column 3.

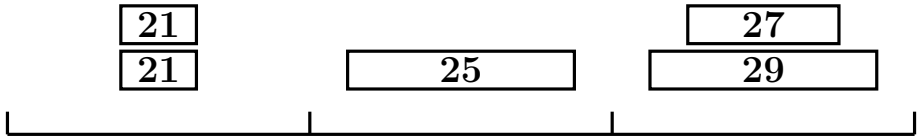
Move 2 3



Move 1 3



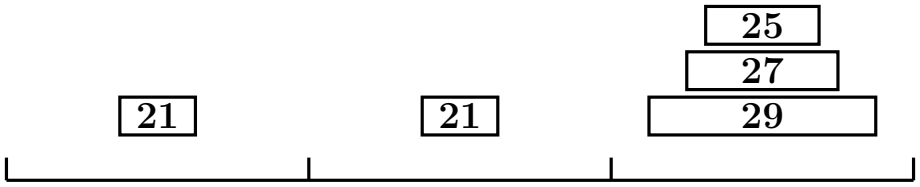
Move 2 1



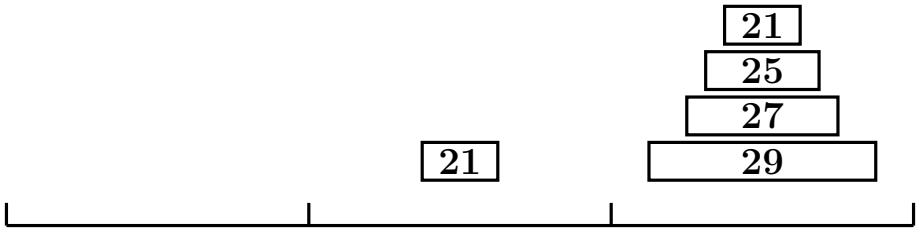
Move 2 3



Move 1 2



Move 1 3



Move 2 3



Move 0 0

We have solved the problem.

[I] Cards

```
Time limit:   1 second
Memory limit: 65535 kBytes
```

Description

The children gathered for an afternoon of games, each bringing their own deck of cards. All decks are identical, and each contains N distinct cards.

At the end of the day, the parents collected all the cards into a single pile. As often happens, some cards may have been lost, and some unfamiliar cards (not belonging to any of the original decks) may have been mixed in during play. The parents want to buy replacements for any missing cards so that each child can have a complete deck again. Help them determine how many new cards they need to purchase.

Input

The first line contains the number of test cases T.
Each test case contains the following data:

- K, N, M are three integers
  - K – the number of children, i.e., the number of complete decks the parents want to put together.
  - N – the number of different cards in the deck.
  - M – the number of cards in the pile.
- The next line contains the cards in the deck (N cards) separated by a space.
- The next line contains the M cards in the pile separated by a space.

Output

A single integer for each test case, the number of additional cards required to make K complete decks.

Constraints

- $1 \leq T \leq 10$
- $1 \leq K \leq 10^9$
- $1 \leq N \leq 10^3$
- $1 \leq M \leq 10^5$

- Each card is represented by a string of up to 10 characters, containing only letters from the English alphabet.

Example

Input

```
3
3 2 4
a b
a a b a
2 3 10
cat dog fish
cat cat cat dog fish dog fish dog fish cat
3 2 5
car ship
train ship car car bus
```

Output

```
2
0
3
```

[J] Firewall Ruleset Minimization

Time limit: 1 second
Memory limit: 65535 kBytes

Description

You are responsible for maintaining the firewall of a high-security data center. The firewall operates based on a ruleset - a prioritized list of rules that decide whether to allow or deny traffic based on IP address ranges and port ranges.

Each rule specifies an action (`ALLOW` or `DENY`), a range of IPv4 addresses, and a range of network ports. The firewall evaluates incoming traffic against the rules from top to bottom, applying **the first matching rule**.

Over time, the ruleset has grown large and contains redundant or overlapping rules. Your task is to remove **redundant rules** without changing the firewall's behavior.

We say that a rule R_j is redundant if there exists an *earlier* single rule R_i (with $i < j$) such that every (IP, port) pair matched by R_j is also matched by R_i.Removing R_j does not change which rule the firewall selects for any (IP, port) pair in this case.

Note: if a rule is covered only by the union of several earlier rules (no single earlier rule fully covers it), it is **not** considered redundant and must not be removed.

Futher remarks

- The firewall evaluates rules from top to bottom. The first rule that matches a given (IP, port) pair determines the action.
- IP ranges are inclusive.
- IPv4 clarification: an address of the form `A.B.C.D` represents a 32-bit unsigned integer with `A` as the most-significant 8 bits and `D` as the least-significant 8 bits. Thus, ranges like `A.B.C.D` to `E.F.G.H` should be interpreted as the closed interval of the corresponding 32-bit integers.

Input

The first line contains a single integer N, the number of firewall rules.

Each of the next N lines describes one rule: `action ip_start ip_end port_start port_end`. The action can either be `ALLOW` or `DENY` (in uppercase). IPv4 addresses are in dot-decimal notation, defining a closed inclusive range of IPs. Ports are integers defining an inclusive port range.

Output

The output consists of the list of rules after removing redundant rules. Each rule is formatted as in the input.

Constraints

- $1 \leq N \leq 10\,000$
- IP addresses are valid IPv4 in dot-decimal notation: `A.B.C.D`, $0 \leq$ `A,B,C,D` ≤ 255.
- Port ranges are inclusive and within 0 to 65535.

Example

Input 1

```
4
ALLOW 192.168.1.0 192.168.1.255 20 25
ALLOW 192.168.1.0 192.168.1.255 26 30
DENY 10.0.0.0 10.0.0.255 0 1023
DENY 10.0.0.0 10.0.0.255 1024 65535
```

Output 1

```
ALLOW 192.168.1.0 192.168.1.255 20 25
ALLOW 192.168.1.0 192.168.1.255 26 30
DENY 10.0.0.0 10.0.0.255 0 1023
DENY 10.0.0.0 10.0.0.255 1024 65535
```

There are no rules that are shadowed by other rules in this example, so all rules are kept.

Input 2

```
3
ALLOW 10.0.0.0 10.0.0.255 80 90
DENY 10.0.0.0 10.0.0.255 80 80
ALLOW 10.0.0.0 10.0.0.255 443 443
```

Output 2

```
ALLOW 10.0.0.0 10.0.0.255 80 90
ALLOW 10.0.0.0 10.0.0.255 443 443
```

The `DENY` rule is removed because the preceding `ALLOW` rule already handles all traffic it would match. Firewalls apply the first matching rule, so the `DENY` is unreachable.

Input 3

```
2
ALLOW 0.0.10.0 0.0.12.0 1000 2000
ALLOW 0.0.11.0 0.0.11.255 1500 1500
```

Output 3

```
ALLOW 0.0.10.0 0.0.12.0 1000 2000
```

The second rule is redundant because every (IP, port) pair it matches is already matched by the earlier `ALLOW` rule.

[K] The Most Secure PIN

Time limit: 1 second
Memory limit: 65535 kBytes

Description

The Bank of Kaprekar City has asked its customers to provide new PIN codes for their bank accounts.

PIN codes are nonnegative integers (without leading zeros) consisting of 1, 2, 3, 4, 5, or 6 decimal digits. A customer may propose multiple PIN codes and the bank will select the first strongest one using its own algorithm, which will then become the customer's new PIN code.

To determine the strength, the bank uses an iterative algorithm. In each iteration, the algorithm takes the given number (the PIN code in the first step), rearranges its digits first in descending and then in ascending order, and calculates the difference between the two resulting numbers. The algorithm then continues by processing this difference in the next iteration. This process repeats until the differences begin to form a repeating pattern.

The strength of a PIN code is the number of integers (including the starting PIN) processed by the bank's algorithm up to and including the first occurrence of a difference that will later repeat in the sequence.

Let us select the strongest PIN code from those provided by customers!

Input

A customer's input file contains the PIN codes they have listed, one per line.

Output

The output should contain one line: the strongest PIN code and its strength (the number of iterations), separated by a space.

Constraints

Each test file contains at most 5000 PIN codes.

PIN codes may consist of 1, 2, 3, 4, 5, or 6 decimal digits.

Examples

Input	Output
8 12 6584 54274 185	6584 8

Explanation

PIN	Strength
8	2
12	3
6584	8
54274	5
185	4

For example, for 6584, the bank's algorithm proceeds as follows:
Step 1: 6584
Step 2: 8654 - 4568 = 4086
Step 3: 8640 - 468 = 8172
Step 4: 8721 - 1278 = 7443
Step 5: 7443 - 3447 = 3996
Step 6: 9963 - 3699 = 6264
Step 7: 6642 - 2466 = 4176
Step 8: 7641 - 1467 = 6174
Step 9: 7641 - 1467 = 6174
On step 9, the result from step 8 repeats, so the strength is 8.

[L] Lost in the Vault

```
  Time limit:  2 second
Memory limit:  65535 kBytes
```

Description

You come to your senses in a dim, humming underground vault. The corridors twist endlessly, some blocked by heavy metal doors marked with a key symbol. Somewhere in the maze lies a single key; once you find it, the locked doors will yield. Your goal: reach the exit in as few steps as possible. If the exit can't be reached, the vault wins.

The map is a rectangular grid. Each cell is one of:

- `S` — your starting cell (exactly one)
- `E` — the exit cell (exactly one)
- `K` — a single key (exactly one)
- `D` — a locked door (can only be passed if you already have the key)
- `#` — an impassable wall
- `.` — an empty cell you can walk on

You can move in the four cardinal directions (up, down, left, right) by one cell per step. Stepping onto the key `K` gives you the key permanently; after that you may move through `D` cells. Walls `#` block movement.

Find the minimum number of steps required to reach the exit `E` from the start `S`. If the exit is unreachable, print `-1`.

Input

The first line contains two integers N and M ($1 \leq N, M \leq 1000$), the number of rows and columns of the grid.

Then follows N lines, each containing M characters describing the map.

The map contains exactly one `S`, exactly one `E`, and exactly one `K`.

Output

Print a single integer: the minimum number of steps to reach `E` from `S`, or `-1` if it is impossible.

Example

Input

```
5 6
S.....
..K...
..D...
#.....
#...E.
```

Output

```
8
```

Explanation

One shortest path is: $(0,0) \to (0,1) \to (0,2) \to (1,2)$ [pick up key] $\to (2,2)$ [pass door] $\to (3,2) \to (4,2) \to (4,3) \to (4,4)$. Total moves $= 8$.

It is not necessary to pick up the key if there's a shortest path to the exit that does not pass any of the doors. In the example, the path $(0,0) \to (0,1) \to (1,1) \to (2,1) \to (3,1) \to (4,1) \to (4,2) \to (4,3) \to (4,4)$ is a valid shortest path too.

[M] Mission: The Odd Code Protocol

Time limit: 1 second
Memory limit: 65535 kBytes

Description

You are Agent ECN, elite field operative of the Global Cipher Intelligence Agency (GCIA). During a covert operation, you intercepted a mysterious data transmission — a sequence of `N` positive integers. Agency analysts suspect the message conceals encrypted key pairs: two numbers form a valid key pair if their sum is odd. Your mission: determine how many such pairs exist in the sequence. Act fast, Agent — the enemy is already triangulating your location.

Input

The first line contains an integer ($1 < N < 1,000,000$)— the number of elements in the sequence. The second line contains the sequence of positive integers (each integer not larger then 1,000,000), separated by spaces.

Output

Print a single integer — the number of pairs you are looking for.

Example

Input

```
6
1 2 4 7 9 2
```

Output

```
9
```

Explanation

The pairs with an odd sum are: (1,2), (1,4), (1,2), (2,7), (2,9), (4,7), (4,9), (7,2), (9,2).

[N] The Library of the Future

Time limit: 2 second
Memory limit: 262144 kBytes

Description

In a modern digital library, new books are uploaded daily across various categories (e.g., literature, science, technology). The library management wishes to analyze which categories experienced the greatest growth in their book collection over a given time interval. The following information is available about the library's operations:

- Each day, a certain number of books is added to exactly one category.
- For given queries, you must determine which category had the largest increase in the number of books during the specified time interval. If multiple categories show equal growth, select the one with the smallest category index.

Input

The first line contains two integers:

- N – the number of days ($1 \leq N \leq 3\,000$)
- Q – the number of queries ($1 \leq Q \leq 3\,000$)

The second line contains N integers, where the i-th number (C_i) represents the index of the category ($1 \leq C_i \leq N$) to which books were added on the i-th day.

The third line contains N integers, where the i-th number (V_i) specifies how many books were added to category C_i on the i-th day ($1 \leq V_i \leq 10^5$).

This is followed by Q lines, each containing two integers: l_i r_i ($1 \leq l_i \leq r_i \leq N$), representing the start and end days of the time interval.

Output

The output should consist of Q lines. Each line should contain one integer: the index of the category that exhibited the greatest cumulative growth during the $[l_i, r_i]$ time interval. If multiple categories show equal growth, output the category with the smallest index.

Example

Input	Output
7 8 1 1 2 2 3 3 1 10 2 5 5 17 3 8 1 1 1 2 1 6 2 3 4 5 5 5 6 7 7 7	1 1 3 2 3 3 1 1

[O] Number Sequences and Laws

```
Time limit:   2 second
Memory limit: 262140 kBytes
```

Description

Mathematics has many laws that relate to different number sequences. One of them is Benford's law. Benford's law describes the relative frequency distribution for **leading digits** of numbers in many naturally occurring datasets.

If leading digits 1-9 had an equal probability, they would each occur about 11.1% of the time. However, Benford's law states that in many real-world sequences, approximately 30% of numbers start with 1, while less than 5% start with 9. According to this law, leading 1s appear roughly 6.5 times as often as leading 9!

This law has practical applications. For instance, it helped computer scientists uncover a bot network on social media. This was possible because, for most genuine users, the number of followers that their followers have adheres to Benford's law, whereas artificial accounts significantly veer from this pattern. Furthermore, Benford's law can be applied to detect if a country manipulates macroeconomic data, among other uses. However, it is important to note that many datasets do not conform to Benford's law.

Your task is to determine if the Fibonacci sequence complies with Benford's law. To do this, write a program that determines the first digit of each number in an n-element Fibonacci sequence and calculates the distribution of these first digits.

Input

The input file contains several lines (at most 8). Each line contains a positive integer n ($n \leq 10\,000\,000$), which defines the length of the Fibonacci sequence to analyze ($F_1 = 1, F_2 = 1, F_3 = 2, \ldots, F_n$).

Output

For each n, output:

- A line containing the value of n.
- A line for each digit from 1 to 9 consisting of:
  - The digit (right-aligned in 4 spaces), followed by a space and '|',
  - The count of numbers starting with that digit in the sequence (right-aligned in 10 spaces), followed by a space and '|',
  - The relative frequency of that digit, formatted to 2 decimal places (right-aligned in 9 spaces, including the decimal point and two digits).
- A blank line after each set of results.

Example

Input

```
5
10
```

Output

```
5
   1 |         2 |    40.00
   2 |         1 |    20.00
   3 |         1 |    20.00
   4 |         0 |     0.00
   5 |         1 |    20.00
   6 |         0 |     0.00
   7 |         0 |     0.00
   8 |         0 |     0.00
   9 |         0 |     0.00

10
   1 |         3 |    30.00
   2 |         2 |    20.00
   3 |         2 |    20.00
   4 |         0 |     0.00
   5 |         2 |    20.00
   6 |         0 |     0.00
   7 |         0 |     0.00
   8 |         1 |    10.00
   9 |         0 |     0.00
```

[P] Resistors for Mike

```
Time limit:   1 second
Memory limit: 65535 kBytes
```

Description

Given n classes of resistors, an old electronics engineer named Mike was asked to calculate the equivalent resistance of a given circuit. The circuit is formed by connecting some of the given resistors in series or in parallel.

Mike is busy and he also lost his glasses, so he asked you to help him with this task.

Input

The input file consists of:

- A line containing n – the number of resistor classes.
- A line containing the resistor values $r_0, \ldots, r_{n-1}$ in Ohms (each between 1.0 and 10000.0 Ohms, inclusive).
- A line with a string expression describing how resistors are connected (series, parallel, or a single resistor).

More formally, an exression may be:

- A single resistor r_j.
- An $S(\ldots)$ expression, representing serial connection between the exressions listed as arguments.
- A $P(\ldots)$ expression, representing parallel connection between the expressions listed as arguments.

Note that nested exressions are allowed. The rules for calculating the resistance of serial and parallel connections can be found in the Remarks section below.

Some examples of valid expressions:

- `S(r0,r1,r2)`
- `P(r3,r4)`
- `S(P(r0,r1),r2)`

Output

On the first line, output the equivalent resistance rounded to 6 digits after the decimal point.

On the second line, print a single integer describing the resistor usage in the circuit:

- **0** – at least one resistor unused, and no resistor used more than once
- **1** – all n resistors used exactly once
- **2** – all resistors used, but some used more than once
- **3** – not all resistors used, and some used more than once

Constraints

- $1 \leq n \leq 30$
- $1.0 <= r_i <= 10000.0$ for each $i = 0, \ldots, n-1$.
- The length of the string expression is at most 150 characters.

Remarks

- A series combination of k components is represented as $S(r_0, r_1, ..., r_{k-1})$ and its equivalent resistance is $\sum_{i=0}^{k-1} r_i$.
- A parallel combination of k components is represented as $P(r_0, r_1, ..., r_{k-1})$ and its equivalent resistance is given by $\frac{1}{\sum_{i=0}^{k-1}\left(\frac{1}{r_i}\right)}$.
- If $n = 2$, $r_0 = r_1 = 1$ Ohm, and the expression is `S(r0,r1)`, then both resistors are considered used exactly once even though their values match. So the second output line should be 1 in this case.

Examples

Input	Output
5 4.0 7.2 3. 1.0 2025.11 S(r0,r1,r2,r3,r4)	2040.310000 1
4 4 4 6 6 S(P(r0,r1),P(r2,r3))	5.000000 1
4 4 6 6 4 S(P(r0,r1),P(r2,r3))	4.800000 1
4 4 4 6 6 P(S(r0,r2),S(r1,r3))	5.000000 1
3 1 2 3.5 S(P(P(r1,r1),r0),r2)	4.000000 2
5 4 4 6 6 9 P(S(r0,r2),S(r1,r3))	5.000000 0
2 3 2 S(P(r1,r1),r0)	4.000000 2
4 1 2 3 4 S(P(r1,r1),r2)	4.000000 3
3 1 2 3 r0	1.000000 0
10 1 2 3 4 5 6 7 8 9 10 S(r0,r1,r2,r3,P(r4,r5,r6),P(r7,S(r8,r9)))	17.592246 1